

Taller — Complejidad algorítmica y Arreglos

SD1004 · Estructura de Datos

Parte 1 — Complejidad algorítmica (Big O)

1.1 Clasifica la complejidad

- **Diccionario físico (por la mitad): $O(\log n)$.** Al dividir las páginas a la mitad en cada paso, se descarta la mitad de los elementos de una sola vez.
- **Caja de carnés (uno por uno): $O(n)$.** En el peor de los casos toca revisar todos los n carnés uno tras otro. Si se duplican los carnés, el tiempo también se duplica.
- **Mazo de cartas (sacar la primera): $O(1)$.** Es un acceso directo al primer elemento. Coger la carta superior siempre toma el mismo tiempo, así el mazo tenga 10 o 10,000 cartas.
- **Comparar estudiantes (cumpleaños): $O(n^2)$.** Para cada uno de los n estudiantes, se debe recorrer otra vez a los otros n estudiantes. Es un proceso de parejas ($n \times n$).
- **Consultar la hora en tu reloj: $O(1)$.** Solo se mira el reloj una vez; no depende de ninguna cantidad de datos.

1.2 De la vida real al análisis

R// Lavar una fila de platos sucios a mano uno por uno. Si hay n platos, tomo cada plato, lo enjabono, lo enjuago y lo pongo a secar. Si en lugar de 5 platos tengo 50 platos (10 veces más), me va a tomar exactamente 10 veces más tiempo lavar toda la fila.

1.3 ¿Cuál escala mejor?

R// Forma B: $O(n \log n)$. Aunque para un n muy pequeño ($n=5$) la Forma A pueda ser más rápida por diferentes cosas, a medida que n crece a miles o millones de datos, $O(n^2)$ crece de forma cuadrática (ejemplo: $1,000^2 = 1,000,000$ operaciones), mientras que $O(n \log n)$ crece mucho más despacio (ejemplo: $1,000 \times 10 = 10,000$ operaciones). La Forma B escala infinitamente mejor para cantidades de datos que no se saben o grandes.

1.4 Verdadero o falso

a. Falso: $O(1)$ significa que el tiempo es constante según el tamaño de la entrada, pero la operación constante podría ser muy pesada (ejemplo: esperar 10 segundos) mientras que una operación $O(n)$ para $n=2$ podría demorarse 2 milisegundos.

b. Verdadero: Big O mide el crecimiento del algoritmo respecto al volumen de datos (n), no los segundos o milisegundos exactos del reloj.

c. Falso: Entrar por índice arreglo[5] es $O(1)$ porque el computador calcula matemáticamente la posición exacta en memoria en un solo paso, sin tener que recorrer los índices 0, 1, 2, 3, 4.

Parte 2 — Arreglos 1D

2.1 Diseña el arreglo

Índice	[0]	[1]	[2]	[3]	[4]	[5]
Valor	3.5	4.2	2.8	5.0	3.9	4.5

R// Su valor es 2.8 y se encuentra en el índice [2].

R// nota_ultimo = notas[5]

2.2 Direcciones de memoria

Cálculos:

notas[0]: $0x2000 + (0 \times 4) = 0x2000$

notas[3]: $0x2000 + (3 \times 4) = 0x2000 + 12 \text{ bytes} = 0x200C$

notas[5]: $0x2000 + (5 \times 4) = 0x2000 + 20 \text{ bytes} = 0x2014$

R// Entrar a notas[3] tarda lo mismo que a notas[0] porque la computadora no recorre puesto por puesto, simplemente aplica la fórmula matemática de multiplicación y suma en un solo ciclo para ir directamente a esa dirección de memoria RAM.

2.3 Búsqueda lineal vs. binaria

Lineal: Paso 1 al 5 revisa del índice 0 al 4 y no son. En el Paso 6 compara con índice 5 (31) y lo encuentra. 6 comparaciones.

Binaria: Paso 1: El centro entre 0 y 7 es índice 3 (23). Como $31 > 23$, buscamos a la derecha. Paso 2: El centro entre 4 y 7 es índice 5 (31). Encontrado. 2 comparaciones.

R// Porque la búsqueda binaria se basa en descartar la mitad superior o inferior según si el valor buscado es mayor o menor que el del centro. Si los datos están desordenados, no se puede saber en cuál mitad está el número.

R// Utilizaría la Búsqueda Binaria. La búsqueda lineal $O(n)$ podría necesitar hasta 1,000,000 de comparaciones, mientras que la binaria $O(\log n)$ solo va a necesitar como máximo aprox 20 comparaciones.

2.4 Insertar un elemento

Si hay espacio: Es muy económico $O(1)$, solo se pone el valor en el índice disponible.

Si no hay espacio: Es costoso $O(n)$, hay que crear un arreglo nuevo más grande y copiar

todos los elementos uno por uno.

R// Es costoso $O(n)$. Se deben mover hacia la derecha todos los elementos mayores que 21 (el 23 y el 27) para abrir el espacio donde irá el 21.

R// Al final con espacio: $O(1)$. En la mitad o redimensionar: $O(n)$.

Parte 3 — Arreglos 2D

3.1 Diseña la matriz

FILA / COLUMNA	COLUMNA 0	COLUMNA 1	COLUMNA 2	COLUMNA 3
FILA 0	salón[0][0]	salón[0][1]	salón[0][2]	salón[0][3]
FILA 1	salón[1][0]	salón[1][1]	salón[1][2]	salón[1][3]
FILA 2	salón[2][0]	salón[2][1]	salón[2][2]	salón[2][3]

R// estado = salón[2][3]

R// Esta matriz tiene $3 \times 4 = 12$ casillas. Fórmula general: Total = filas x columnas.

3.2 De 2D a memoria (row-major)

Aplanamiento: En la RAM quedan las 12 casillas seguidas en una sola línea, primero toda la fila 0, luego toda la fila 1 y luego la fila 2.

[0][0]	[0][1]	[0][2]	[0][3]	[1][0]	[1][1]	[1][2]	[1][3]	[2][0]	[2][1]	[2][2]	[2][3]
--------	--------	--------	--------	--------	--------	--------	--------	--------	--------	--------	--------

Dirección [1][2]: $0x1000 + ((1 \times 4) + 2) \times 4 = 0x1000 + 24 \text{ bytes} = 0x1018$

Otras direcciones:

[0][0]: $0x1000 + ((0 \times 4) + 0) \times 4 = 0x1000$

[2][3]: $0x1000 + ((2 \times 4) + 3) \times 4 = 0x1000 + 44 \text{ bytes} = 0x102C$

3.3 Recorrido con ciclos anidados

Pseudocódigo:

ocupados = 0

Para f desde 0 hasta filas - 1 Hacer:

```

    Para c desde 0 hasta columnas - 1 Hacer:
        Si salon[f][c] == 1 Entonces:
            ocupados = ocupados + 1
        FinSi
    FinPara
FinPara
Mostrar ocupados

```

R// $O(n)$ donde n es el número total de casillas. Hay que revisar cada casilla individualmente exactamente una vez.

R// Si las filas se duplican ($2f$) y las columnas también ($2c$), la cantidad total de casillas se multiplica por 4 ($2f \times 2c = 4fc$). Entonces, el tiempo de ejecución se multiplicará por 4.

3.4 Conectando todo

R// Aprender sobre Big O es bueno porque nos ayuda a saber qué estructura usar según lo que se necesite hacer. Por ejemplo, en un arreglo normal 1D es muy fácil entrar a una casilla directa si sabemos la posición, pero si buscamos un dato de uno en uno se puede demorar mucho, a menos que los datos estén organizados y se pueda usar la búsqueda binaria para ir más rápido. Con los arreglos 2D pasa algo parecido: sirven mucho para organizar cosas en filas y columnas, pero si se necesitan revisar todas las casillas ya toca usar dos ciclos unidos y eso hace que se demore más. En resumen, entender cómo la computadora guarda todo esto en la memoria nos ayuda a usar la mejor opción para que nuestro programa no se ponga lento.